

Property Data Engineering Sprint — Click-by-Click Guide

SSIS in Visual Studio 2022 + SQL Server | Version 2 (As-Built, July 2026)

This is the updated version of the original guide, revised to match how the project was **actually built and debugged**. Everything marked **UPDATED** reflects a change made during the real build of PKG1 and PKG2. Both packages now execute successfully end-to-end.

Environment (as built)	
SQL Server instance	SIRILENOVO\IC
Database	PropertyDW
Project folder	C:\Users\drsre\OneDrive\Desktop\IC Internship\Tasks\BI Competition - Data Warehouse Modeling & Creation\PropertySprint_ETL\
Packages	PKG1_Load_Staging.dtsx PKG2_Load_StarSchema.dtsx
Final row counts (staging)	load_Schools 2,211 load_PropertyMedianValue 4,674 load_AusLocation 16,078
Final row counts (facts)	FactPropertyValue 3,921 FactSchoolLocation 2,168

■ When creating any OLE DB connection: the server name is **SIRILENOVO\IC** and the database **PropertyDW** is chosen separately from the dropdown. Do NOT type SIRILENOVO\IC\PropertyDW as the server name — that caused SQL Network Interfaces Error 26 during the build.

PHASE 0 — One-time prerequisites

0.1 Check SQL Server is installed

1. Press Windows key, type **SSMS**, open SQL Server Management Studio.
2. Connect to Server dialog: Server type = Database Engine, **Server name** = **SIRILENOVO\IC**, Authentication = Windows Authentication.
3. Click **Connect**.

0.2 Excel driver (ACE OLEDB)

UPDATED: Both **Microsoft.ACE.OLEDB.12.0** and **16.0** were verified installed on this machine via PowerShell ((New-Object system.data.oledb.oledbenumerator).GetElements() | Select SOURCES_NAME). No re-install needed. If setting up on a new machine, install the Microsoft Access Database Engine 2016 Redistributable (64-bit).

0.3 Confirm SSIS extension in Visual Studio 2022

1. Open VS 2022 → Continue without code → Extensions → Manage Extensions → search **Integration Services**.
2. **SQL Server Integration Services Projects 2022** should show Installed.

PHASE 1 — Create the Data Warehouse in SSMS

1. Open SSMS → connect to **SIRILENOVO\IC**.
2. File → Open → File... → **01_PropertyDW_Setup.sql** → press **F5**.
3. Refresh Databases → expand **PropertyDW** → **Tables**. Expect: load_Schools, load_PropertyMedianValue, load_AusLocation (staging); DimLocation, DimDate, DimCategory, DimSchool; FactPropertyValue, FactSchoolLocation.

UPDATED: Staging table columns are all **NVARCHAR** and were widened to **255–500** characters during the build (e.g. school_code is NVARCHAR(255), not 50). The original 50-char assumption caused truncation warnings and unicode/non-unicode errors — the live table definitions are the source of truth, not the first-draft script.

UPDATED: DimDate has NO identity column. DateKey is a calculated YYYYMM integer (e.g. 201701). This matters for the Reset Warehouse task in Phase 4.

DimCategory seed — corrected version:

UPDATED: The original draft produced **8 duplicate rows** (two seed scripts both inserted without clearing first) and used a NULL upper bound on the top band, which breaks BETWEEN-style range checks. The corrected seed clears the table first and inserts exactly 4 rows with non-overlapping boundaries and a large fixed ceiling:

```
DELETE FROM dbo.DimCategory;
DBCC CHECKIDENT ('dbo.DimCategory', RESEED, 0);

INSERT INTO dbo.DimCategory (CategoryName, MinValue, MaxValue) VALUES
(' $0-$750K', 0, 750000.00),
(' $750K-$1.5M', 750000.01, 1500000.00),
(' $1.5M-$2.5M', 1500000.01, 2500000.00),
(' $2.5M+', 2500000.01, 99999999.99);
```

Verify: right-click dbo.DimCategory → Select Top 1000 Rows → exactly 4 rows, CategoryKey 1–4.

PHASE 2 — SSIS project + Package 1 (Staging Load)

2.1 Create the project

1. VS 2022 → Create a new project → search **Integration Services** → Integration Services Project → Next.
2. Project name: **PropertySprint_ETL**. Location: the BI Competition task folder (OneDrive path above). Create.
3. In Solution Explorer, rename Package.dtsx → **PKG1_Load_Staging.dtsx** → Yes on the rename prompt.

■ **Adding an existing .dtsx to the project?** If the file already sits inside the project folder, Add → Existing Package creates a duplicate named “PKG1_Load_Staging (1).dtsx”. Fix: move the file OUT to the parent folder first, then Add Existing Package — VS copies it back in cleanly under the correct name.

2.2 Fix the 64-bit Excel issue up front

1. Solution Explorer → right-click project **PropertySprint_ETL** → Properties → Configuration Properties → Debugging.
2. Set **Run64BitRuntime = False** → OK.

2.3 Connection Managers (4 total)

OLE DB (warehouse):

1. Right-click Connection Managers pane → New OLE DB Connection... → New...
2. Server or file name: **SIRILENOVOIC** (server only — never append the database name). Authentication: Windows. Database: **PropertyDW** from the dropdown.
3. Test Connection → OK → OK. Rename to **CM_PropertyDW**.

Flat File (schools CSV) — CM_Schools_CSV:

1. New Flat File Connection... → Browse to NSW-Public-Schools-Master-Dataset-07032017.csv (file type: CSV files).
2. Tick **Column names in the first data row**. Format: Delimited. Text qualifier: " (double-quote).
3. Columns page — verify the grid splits correctly.

UPDATED: Advanced page: set **OutputColumnWidth = 500** for the long text columns (school_name, school_email, street, facs_district and similar). The original guide said 255, but row 1638 of the CSV still truncated at 255 — 500 is the value that finally worked. Leave everything as **DT_STR** here; the unicode conversion happens next in the destination/staging (NVARCHAR) mapping.

2.4 Control Flow — truncate step

1. Drag **Execute SQL Task** onto the Control Flow canvas. Name: **SQL - Truncate Staging**. Connection: CM_PropertyDW. SQLSourceType: Direct input.

```
TRUNCATE TABLE dbo.load_Schools;  
TRUNCATE TABLE dbo.load_PropertyMedianValue;  
TRUNCATE TABLE dbo.load_AusLocation;
```

■ Paste plain SQL only — during the build a stray markdown backtick (`) pasted into the SQLStatement box caused a syntax failure. Check the first and last characters of the statement box.

2.5 Three Data Flow Tasks

1. Drag three **Data Flow Tasks: DFT - Load Schools, DFT - Load PropertyValue, DFT - Load AusLocation**.
2. Connect the green arrow from SQL - Truncate Staging to each of the three DFTs (they run in parallel after the truncate).

2.6 Inside DFT - Load Schools

1. Flat File Source → CM_Schools_CSV → tick **Retain null values**.
2. OLE DB Destination → CM_PropertyDW → Table or view - fast load → **[dbo].[load_Schools]** → Mappings auto-map by name.

UPDATED: If you add a Data Conversion transformation here, every output column must be **DT_WSTR** (unicode) because the staging tables are NVARCHAR. During the build, school_code was accidentally flipped to DT_STR, which produced the “cannot convert between unicode and non-unicode string data types” validation error. Also match lengths to the **live** table (school_code = 255).

2.7 Inside DFT - Load PropertyValue

1. Excel Source → CM_PropertyValue_XLSX → sheet '**NSW_PropertyMedainValue.csv**' (yes, the sheet is named like a csv).
2. Data Conversion → tick every input column → all **string [DT_WSTR]**, length 100 (50 for Postcode/Year/Month).
3. OLE DB Destination → fast load → **[dbo].[load_PropertyMedianValue]** → map the “Copy of ...” columns to the staging columns.

2.8 Inside DFT - Load AusLocation

1. Excel Source (CM_AusLocation_XLSX, sheet '**AUS_Post_suburb v2**') → Data Conversion (all DT_WSTR) → OLE DB Destination → **[dbo].[load_AusLocation]**.
2. Mappings: source column **District** (typo in the source file) maps to staging column **district**.

UPDATED: A residual truncation **warning** on state/state_code is cosmetic and can be accepted: the ACE OLEDB Excel driver always reports text columns as 255 characters regardless of actual content. The package still runs green.

■ If the Excel connection suddenly fails with a lock/access error and nothing else has changed: the project lives in a **OneDrive-synced folder** and a sync lock can hold the .xlsx. A computer restart cleared this during the build.

2.9 Run Package 1

1. Press **F5**. All three DFTs should go green.
2. Verify in SSMS — expected counts: **load_Schools = 2,211, load_PropertyMedianValue = 4,674, load_AusLocation = 16,078**.

UPDATED: If VS loops on “the package has been modified since the last build” and refuses to run, bypass the debugger and run from PowerShell with dtexec:

```
& "C:\Program Files\Microsoft SQL Server\160\DTS\Binn\DTExec.exe" `  
/File "C:\Users\drsre\OneDrive\Desktop\IC Internship\Tasks\  
BI Competition - Data Warehouse Modeling & Creation\  
PropertySprint_ETL\PropertySprint_ETL\PKG1_Load_Staging.dtsx" `  
/X86
```

(path shown wrapped for readability - keep it on ONE line in PowerShell)

Result should end with **DTSER_SUCCESS**.

PHASE 3 — Dimensional model documentation

The 1-page design doc (Bus Matrix, schema choice, data quality decisions) was expanded into **four Word documents** for submission: SQL Scripts, Bus Matrix, Schema Diagram (with embedded star schema image), and Design Decisions — all written in plain, student-friendly language.

Bus Matrix:

Business Process	DimLocation	DimDate	DimCategory	DimSchool
Property valuation	X	X	X	
School presence	X			X

Schema choice: Star schema. DimLocation is the conformed dimension shared by both facts. **FactPropertyValue** (grain: one row per suburb per month) carries the median value measure and a CategoryKey.

FactSchoolLocation is a factless fact table — no measures; it records that a school exists in a location, enabling counts like “number of selective schools in \$2.5M+ suburbs”.

Data quality decisions documented: 'np' placeholder text in numeric school columns → NULL; inconsistent state labels; leading spaces in AUS state names; trailing spaces in suburb names; the 'Distic' column typo; District/Location mostly empty in the property file; and the exclusion counts from the fact loads (see Phase 4.9).

PHASE 4 — Package 2 (Transform + Load the Star Schema)

4.1 Create the package

1. Solution Explorer → right-click SSIS Packages → New SSIS Package → rename **PKG2_Load_StarSchema.dtsx**.
2. Recreate **CM_PropertyDW** in this package (package-level connections don't carry over), or convert PKG1's connections to Project Connections so all packages share them.

4.2 Control Flow layout (final, as built)

UPDATED: A **SQL - Load DimCategory** task was added — it wasn't in the original plan but CategoryKey in FactPropertyValue requires DimCategory to be populated first.

```
[SQL - Reset Warehouse]
    ↓
[DFT - Load DimLocation]
    ↓
[DFT - Load DimSchool]
    ↓
[SQL - Load DimDate]
    ↓
[SQL - Load DimCategory]      <-- NEW
    ↓
[DFT - Load FactPropertyValue]
    ↓
[DFT - Load FactSchoolLocation]
```

4.3 SQL - Reset Warehouse (corrected)

UPDATED: Two corrections vs the original draft: (1) DimCategory is now included in the deletes; (2) **DimDate is deleted but NOT reseeded** — it has no identity column (DateKey is calculated YYYYMM), and running DBCC CHECKIDENT against it fails the whole package with “'DimDate' does not contain an identity column”.

```
-- DELETE facts first (FK constraints), then dims
DELETE FROM dbo.FactPropertyValue;
DELETE FROM dbo.FactSchoolLocation;

DELETE FROM dbo.DimLocation;
```

```

DELETE FROM dbo.DimDate;
DELETE FROM dbo.DimCategory;
DELETE FROM dbo.DimSchool;

-- Reset identity counters (DimDate excluded - no identity column)
DBCC CHECKIDENT ('dbo.FactPropertyValue', RESEED, 0);
DBCC CHECKIDENT ('dbo.FactSchoolLocation', RESEED, 0);
DBCC CHECKIDENT ('dbo.DimLocation', RESEED, 0);
DBCC CHECKIDENT ('dbo.DimCategory', RESEED, 0);
DBCC CHECKIDENT ('dbo.DimSchool', RESEED, 0);

```

DELETE (not TRUNCATE) is used because the dimension tables are referenced by foreign keys; ResultSet = None; TimeOut = 0.

4.4 DFT - Load DimLocation

1. OLE DB Source → CM_PropertyDW → Data access mode: SQL command:

```

SELECT DISTINCT
    LTRIM(RTRIM(suburb))           AS Suburb,
    TRY_CAST(postcode AS INT)      AS Postcode,
    LTRIM(RTRIM(city))            AS City,
    LTRIM(RTRIM(district))        AS District,
    LTRIM(RTRIM([state]))          AS [State],
    LTRIM(RTRIM(state_code))       AS StateCode,
    TRY_CAST(lat AS DECIMAL(9,6))  AS Latitude,
    TRY_CAST(lon AS DECIMAL(9,6))  AS Longitude
FROM dbo.load_AusLocation
WHERE suburb IS NOT NULL AND TRY_CAST(postcode AS INT) IS NOT NULL;

```

2. OLE DB Destination → **[dbo].[DimLocation]** → Mappings (LocationKey stays <ignore> — IDENTITY).

UPDATED: If a truncation warning appears on the source's output columns: fix the lengths through **Advanced Editor** (right-click the OLE DB Source → Show Advanced Editor → Input and Output Properties → Output Columns). Editing the SQL in the standard editor did NOT refresh the cached metadata — the Advanced Editor is what worked.

4.5 DFT - Load DimSchool

1. OLE DB Source → SQL command:

```

SELECT
    TRY_CAST(school_code AS INT)           AS SchoolCode,
    LTRIM(RTRIM(school_name))              AS SchoolName,
    LTRIM(RTRIM(town_suburb))              AS Suburb,
    TRY_CAST(postcode AS INT)              AS Postcode,
    level_of_schooling                     AS LevelOfSchooling,
    selective_school                        AS SelectiveSchool,
    opportunity_class                      AS OpportunityClass,
    school_specialty_type                   AS SchoolSpecialtyType,
    school_gender                           AS SchoolGender,
    TRY_CAST(NULLIF(student_number, 'np') AS INT) AS StudentNumber,
    TRY_CAST(NULLIF(ICSEA_Value, 'np') AS INT) AS ICSEA_Value,
    TRY_CAST(latitude AS DECIMAL(9,6))      AS Latitude,
    TRY_CAST(longitude AS DECIMAL(9,6))     AS Longitude
FROM dbo.load_Schools;

```

NULLIF(col,'np') is the fix for the 'np' placeholder problem — call it out in the Design Decisions doc.

2. OLE DB Destination → **[dbo].[DimSchool]** → Mappings → OK.

4.6 SQL - Load DimDate

```

INSERT INTO dbo.DimDate (DateKey, [Year], [Month], MonthName)
SELECT DISTINCT
    TRY_CAST(Updated_Year AS INT) * 100 + TRY_CAST(Updated_Month AS INT),
    TRY_CAST(Updated_Year AS INT),
    TRY_CAST(Updated_Month AS INT),
    DATENAME(MONTH, DATEFROMPARTS(TRY_CAST(Updated_Year AS INT),
                                  TRY_CAST(Updated_Month AS INT), 1))

```

```
FROM dbo.load_PropertyMedianValue
WHERE TRY_CAST(Updated_Year AS INT) IS NOT NULL
      AND TRY_CAST(Updated_Month AS INT) IS NOT NULL;
```

Result as built: 12 rows, DateKey 201701–201712, correct month names.

4.7 SQL - Load DimCategory (new task)

Execute SQL Task → CM_PropertyDW → the corrected seed block from Phase 1 (DELETE + RESEED + 4 INSERTs). Keeping it inside the package makes PKG2 fully re-runnable from scratch — no manual SSMS fix needed.

4.8 DFT - Load FactPropertyValue (as built)

UPDATED: The category logic changed twice during the build and this is the **final** approach: a SQL BETWEEN JOIN against DimCategory was tried first, then replaced with a proper **SSIS Derived Column Transformation** that computes **CategoryKey** directly (an integer 1–4). This satisfies the task's explicit "transformation in SSIS to categorize Median Property Values" requirement, and removes the need for a Category Lookup component entirely.

(a) **OLE DB Source** — SQL command:

```
SELECT
    LTRIM(RTRIM(Suburb))                AS Suburb,
    TRY_CAST(Postcode AS INT)           AS Postcode,
    TRY_CAST(Property_Median_Value AS MONEY) AS PropertyMedianValue,
    TRY_CAST(Updated_Year AS INT) * 100
    + TRY_CAST(Updated_Month AS INT)    AS DateKey
FROM dbo.load_PropertyMedianValue
WHERE TRY_CAST(Property_Median_Value AS MONEY) IS NOT NULL
      AND TRY_CAST(Postcode AS INT) IS NOT NULL;
```

(b) **Derived Column — DER - Assign Category**. Add one row: Derived Column Name = **CategoryKey**, Expression:

```
[PropertyMedianValue] <= 750000 ? 1 :
([PropertyMedianValue] <= 1500000 ? 2 :
([PropertyMedianValue] <= 2500000 ? 3 : 4))
```

These 1–4 values match the CategoryKey identity values seeded in DimCategory (which is why the corrected seed reseeds to 0 and inserts exactly 4 rows — the keys must be stable).

(c) **Lookup — Location key**.

- Connection page: **[dbo].[DimLocation]**.
- Columns page: drag **Suburb** → **Suburb** AND **Postcode** → **Postcode** (two join lines — suburb alone is ambiguous across postcodes). Tick **LocationKey** to return it.
- General page: no-match handling = **Redirect rows to no match output** (unmatched suburbs are diverted instead of failing the run).
- When connecting to the next component, choose **Lookup Match Output**.

(d) **OLE DB Destination** → **[dbo].[FactPropertyValue]** → Mappings: LocationKey, DateKey, CategoryKey, PropertyMedianValue.

4.9 DFT - Load FactSchoolLocation

1. **OLE DB Source** — SQL command:

```
SELECT SchoolKey, LTRIM(RTRIM(Suburb)) AS Suburb, Postcode
FROM dbo.DimSchool;
```

2. Lookup against **[dbo].[DimLocation]**, join Suburb + Postcode, return LocationKey, no-match = **Redirect**.
3. **OLE DB Destination** → **[dbo].[FactSchoolLocation]** → map SchoolKey and LocationKey (Lookup Match Output).

4.10 Run Package 2 — actual results

1. Solution Explorer → right-click PKG2_Load_StarSchema.dtsx → **Set as StartUp Object** → F5.

UPDATED: Verified outcome from the successful run (document these numbers in the Design Decisions doc):

Load	Loaded	Excluded	Why excluded
FactPropertyValue	3,921 of 4,674	752 + 2	752 rows had NULL median values; 2 rows had suburb/postcode not present in DimLocation
FactSchoolLocation	2,168 of 2,211	43	School suburb/postcode not present in the AUS location reference (expected)

PHASE 5 — Verify + package the submission

Run these in SSMS and screenshot the results:

```
-- Fact populated with all keys resolved
SELECT TOP 10 * FROM dbo.FactPropertyValue;

-- Category distribution (proves the bucketing works)
SELECT c.CategoryName, COUNT(*) AS Suburbs,
       MIN(f.PropertyMedianValue) AS MinVal,
       MAX(f.PropertyMedianValue) AS MaxVal
FROM dbo.FactPropertyValue f
JOIN dbo.DimCategory c ON c.CategoryKey = f.CategoryKey
GROUP BY c.CategoryName;

-- The factless fact answering a real business question:
-- how many schools sit in $2.5M+ suburbs?
SELECT COUNT(DISTINCT s.SchoolKey) AS SchoolsInPremiumSuburbs
FROM dbo.FactSchoolLocation fs
JOIN dbo.DimSchool s ON s.SchoolKey = fs.SchoolKey
JOIN dbo.FactPropertyValue f ON f.LocationKey = fs.LocationKey
JOIN dbo.DimCategory c ON c.CategoryKey = f.CategoryKey
WHERE c.CategoryName = '$2.5M+';
```

Submission checklist (as built): the two .dtsx packages (whole VS solution folder), the setup .sql script (with the corrected DimCategory seed and Reset Warehouse block), the **four Word documents** (SQL Scripts, Bus Matrix, Schema Diagram, Design Decisions), and screenshots of green runs + verification queries.

TROUBLESHOOTING — every error actually hit during the build

Error / symptom	Cause	Fix that worked
SQL Network Interfaces Error 26 (server not found)	Database name appended to server name (SIRILENOVOIC\PropertyDW)	Server name = SIRILENOVOIC only; pick PropertyDW separately in the database dropdown
"DimDate' does not contain an identity column" fails PKG2 at Reset Warehouse	DBCC CHECKIDENT run against DimDate, whose DateKey is a calculated YYYYMM value, not IDENTITY	Delete the DBCC CHECKIDENT('dbo.DimDate',...) line; keep the DELETE
DimCategory shows 8 rows instead of 4	Old + new seed scripts both inserted without clearing	DELETE all rows, RESEED to 0, insert the 4 corrected bands
Top price band never matches in range checks	NULL upper bound breaks BETWEEN logic	Use a large fixed ceiling (999999999.99)
"Cannot convert between unicode and non-unicode string data types"	A Data Conversion column flipped to DT_STR while staging tables are NVARCHAR	Set every conversion output to DT_WSTR; verify after any edit
"Text was truncated" on the CSV load (row 1638)	Column width smaller than real data	Flat File Connection → Advanced → raise OutputColumnWidth to 500 for long text columns
Truncation warning persists after fixing the source SQL	The standard editor doesn't refresh cached output metadata	Show Advanced Editor → Input and Output Properties → fix Output Column lengths there
state/state_code truncation warning that never goes away	ACE OLEDB Excel driver always reports text columns as 255	Cosmetic — accept it; package runs green
Excel connection suddenly fails / file locked	OneDrive sync lock on the .xlsx (project is in a synced folder)	Restart the computer to clear the lock

Error / symptom	Cause	Fix that worked
"PKG1_Load_Staging (1).dtsx" duplicate created	Add Existing Package copies the file into a folder where it already exists	Move the .dtsx to the parent folder first, then Add Existing Package
VS loops on "modified since last build" and won't run	VS 2022 SSIS build-state glitch	Run the package directly with DTEXec.exe from PowerShell (/X86)
'Microsoft.ACE.OLEDB.16.0' provider not registered / empty sheet dropdown	Excel driver missing or 64-bit runtime	Install ACE redistributable; set Run64BitRuntime = False
Lookup: cannot insert NULL into FactPropertyValue.LocationKey	Unmatched suburbs going down the match path	Set Location Lookup no-match handling to Redirect rows to no match output
Execute SQL Task fails on valid-looking SQL	Stray markdown backtick pasted into the SQLStatement box	Delete the backtick; paste plain SQL only

This maps almost one-to-one onto skills PL-300 already gave you: Derived Column $\approx$ Power Query custom column, Lookup $\approx$ merge, and the star schema is the same Kimball model Power BI wants anyway.